

UNIVERSIDAD TÉCNICA

Facultad de Ciencias de la Computación

Carrera de Ingeniería en Tecnologías de la Información

Práctica Experimental — Unidad IV

Temas 13–16: Arquitecturas Web, APIs REST,
SOAP y Tendencias Emergentes

Materia: Aplicaciones Web

Grupo: PFC — Pallo · Beltrán · Cruz

PFC: SGED — Sistema de Gestión Estudiantil Deportiva

9 de agosto de 2026

Resumen

El presente informe documenta la Práctica Experimental correspondiente a la Unidad IV de la asignatura Aplicaciones Web, abarcando los Temas 13 a 16. El equipo diseñó, implementó y demostró el **Sistema de Gestión Estudiantil Deportiva (SGED)**, una aplicación web completa basada en el patrón Modelo–Vista–Controlador (MVC) con backend en *Spring Boot 3.2* (Java 21) y frontend en *Angular 21*. Se expone la API REST del sistema con documentación interactiva mediante Swagger UI accesible en `/api/docs`, se ejecutan en vivo al menos tres endpoints representativos y se elabora una comparativa rigurosa entre SOAP y REST con nueve criterios de análisis. Finalmente, se reflexiona críticamente sobre las tendencias emergentes: Jamstack, Progressive Web Apps (PWA) e Inteligencia Artificial generativa aplicada al desarrollo web.

Abstract. This report documents the Experimental Practice for Unit IV of the Web Applications course, covering Topics 13–16. The team designed, implemented, and demonstrated the **Sports Student Management System (SGED)**, a full-stack web application built on the Model–View–Controller (MVC) pattern using a *Spring Boot 3.2* (Java 21) backend and an *Angular 21* frontend. The system’s REST API is presented with interactive documentation via Swagger UI at `/api/docs`; at least three representative endpoints are executed live. A rigorous nine-criterion comparison between SOAP and REST is provided, and the report closes with a critical reflection on emerging trends: Jamstack, Progressive Web Apps (PWA), and generative Artificial Intelligence in web development.

Palabras clave: REST, SOAP, MVC, API, OpenAPI, Swagger, Jamstack, PWA, IA generativa, Spring Boot.

Keywords: REST, SOAP, MVC, API, OpenAPI, Swagger, Jamstack, PWA, generative AI, Spring Boot.

Índice

Resumen	1
1 Introducción	4
2 Revisión Bibliográfica	4
2.1 Fielding (2000) — REST como estilo arquitectónico	4
2.2 Richardson y Ruby (2007) — REST Práctico	5
2.3 SpringDoc OpenAPI 2 — Documentación oficial	5
2.4 W3C (2007) — Especificación SOAP 1.2	5
2.5 Masse (2011) — REST API Design Rulebook	5
2.6 Garriga et al. (2021) — Adopción de APIs en América Latina	6
3 Aplicación MVC Completa — SGED	6
3.1 Descripción general	6
3.2 Arquitectura MVC	6
3.3 Módulos de la Entrega 1B	6
3.4 Verificación en laboratorio	6
4 API REST Documentada — Swagger UI	7
4.1 Configuración de Swagger UI	7
4.2 Endpoints demostrados en vivo	7
4.2.1 Endpoint 1 — POST /api/auth/login	7
4.2.2 Endpoint 2 — GET /api/categorias/activas	8
4.2.3 Endpoint 3 — POST /api/asistencias/qr/sesion/{idSesion}/token	9
4.3 Checklist de ensayo previo al laboratorio	9
5 Comparativa SOAP vs. REST	9
5.1 Tabla de criterios	9
5.2 Ejemplo SOAP y su equivalente REST	9
5.2.1 SOAP — NumberToWords (servicio público verificado con curl)	9
5.2.2 REST — Equivalente en SGED	11
5.3 Análisis del contraste	12

6	Tendencias Emergentes en el Desarrollo Web	12
6.1	Jamstack: desacoplamiento como paradigma de arquitectura	12
6.2	Progressive Web Apps (PWA): cierre de la brecha web/nativa	13
6.2.1	Demo funcional — Service Worker en el SGED	13
6.3	Inteligencia Artificial Generativa: reconfiguración del rol del desarrollador	15
7	Conclusiones	16
8	Trabajo Futuro	16
	Referencias	18
A	Archivo <code>referencias.bib</code>	18

1 Introducción

El ecosistema del desarrollo web ha evolucionado aceleradamente durante la última década. La adopción masiva de arquitecturas orientadas a servicios, el auge de las API REST y la irrupción de paradigmas como Jamstack y la IA generativa obligan al profesional contemporáneo a comprender tanto los fundamentos teóricos como sus implicaciones prácticas.

Esta práctica tiene como propósito consolidar los conocimientos de la Unidad IV a través de la implementación, documentación y análisis crítico de un sistema real: el SGED (*Sports Student Management System*), desarrollado como Proyecto de Fin de Carrera (PFC) del equipo.

El informe se organiza como sigue: la Sección 2 presenta la revisión bibliográfica con al menos cinco fuentes académicas en norma APA; la Sección 3 describe la aplicación MVC completa; la Sección 4 documenta la API REST con Swagger UI; la Sección 5 elabora la comparativa SOAP vs. REST; la Sección 6 reflexiona sobre las tendencias emergentes; y las Secciones 7 y 8 presentan las conclusiones y el trabajo futuro, respectivamente.

2 Revisión Bibliográfica

De acuerdo con la Directriz 1, se consultaron al menos cinco fuentes académicas primarias, incluyendo la disertación seminal de Fielding (2000) sobre REST y la documentación oficial del framework utilizado (SpringDoc Team (2024)).

2.1 Fielding (2000) — REST como estilo arquitectónico

Fielding (2000) definió en su disertación doctoral la arquitectura *Representational State Transfer* (REST) como un conjunto de seis restricciones: cliente-servidor, *stateless*, caché, interfaz uniforme, sistema por capas y código bajo demanda (opcional). Estas restricciones, cuando se aplican de forma coherente, producen sistemas con alta escalabilidad y evolvabilidad (Fielding, 2000). El SGED adopta las cinco restricciones obligatorias: sesiones de usuario almacenadas en el cliente mediante JWT en *cookie HttpOnly* (*stateless*), respuestas cacheables para GET `/api/categorias/activas` e interfaz uniforme basada en sustantivos en plural.

2.2 Richardson y Ruby (2007) — REST Práctico

Richardson y Ruby (2007) ampliaron el trabajo de Fielding con una guía práctica sobre el diseño de APIs RESTful, introduciendo el concepto de *Resource-Oriented Architecture* (ROA) y el modelo de madurez REST por niveles. Destacan que el uso correcto de los verbos HTTP y la hipermedia (HATEOAS) diferencia una API verdaderamente RESTful de una simple API HTTP (Richardson & Ruby, 2007).

2.3 SpringDoc OpenAPI 2 — Documentación oficial

SpringDoc Team (2024) describe la integración de SpringDoc OpenAPI 2 con Spring Boot 3, que es la librería utilizada en el SGED para exponer la especificación OpenAPI 3.1 en `/api/docs.json` y la interfaz Swagger UI en `/api/docs`.

Listing 1: Configuración SpringDoc en `application.yml`

```
springdoc:
  api-docs:
    path: /api/docs.json
  swagger-ui:
    path: /api/docs
    operationsSorter: method
```

2.4 W3C (2007) — Especificación SOAP 1.2

W3C (2007) define el protocolo SOAP 1.2, que establece un modelo de mensajería basado en XML con un *Envelope* obligatorio (Header + Body), independiente del transporte subyacente. La especificación WSDL 2.0, también normativa del W3C, describe el contrato del servicio de forma tipada y *machine-readable*.

2.5 Masse (2011) — REST API Design Rulebook

Masse (2011) sistematiza las buenas prácticas de diseño de APIs RESTful en un conjunto de reglas prescriptivas que cubren nomenclatura de URIs, uso de métodos HTTP, gestión de errores con códigos de estado y versionado. Las reglas de Masse se aplicaron en el SGED: URIs en minúsculas con guiones medios, sustantivos en plural (`/api/categorias`), respuestas de error consistentes.

2.6 Garriga et al. (2021) — Adopción de APIs en América Latina

Garriga et al. (2021) analizan el estado de adopción de APIs REST en proyectos de software en América Latina, concluyendo que Ecuador mantiene servicios SOAP legados en entidades gubernamentales y bancarias por razones de compatibilidad histórica, aunque el sector privado y las *fintechs* han migrado mayoritariamente a REST.

3 Aplicación MVC Completa — SGED

3.1 Descripción general

El SGED es un sistema web para la gestión de estudiantes deportivos, que cubre los módulos de autenticación, categorías, sesiones de entrenamiento y control de asistencia mediante generación y lectura de códigos QR.

3.2 Arquitectura MVC

La aplicación implementa el patrón Modelo–Vista–Controlador en dos capas independientes:

- **Modelo + Controlador (Backend):** Spring Boot 3.2 / Java 21. Los modelos son entidades JPA mapeadas a PostgreSQL 16. Los controladores REST exponen los recursos con `@RestController` y `@RequestMapping("/api/...")`. La capa de servicio (`@Service`) encapsula la lógica de negocio.
- **Vista (Frontend):** Angular 21 (Angular CLI). La comunicación con el backend se realiza mediante el `HttpClient` de Angular con `withCredentials: true` para el manejo automático de cookies.

3.3 Módulos de la Entrega 1B

3.4 Verificación en laboratorio

Para la sesión de laboratorio, el docente verificará la aplicación ejecutando:

1. `make up` (o `arrancar.bat` en Windows) para levantar el stack completo.
2. Navegar a `http://localhost:4200` y autenticarse con las credenciales de `db/seed.sql`.
3. Recorrer los módulos de categorías, sesiones y asistencia QR en tiempo real.

Cuadro 1: Módulos implementados en la Entrega 1B del SGED

Módulo	Backend (Spring Boot)	Frontend (Angular)
Autenticación	AuthController + JWT HttpOnly	Página Login, gestión de sesión
Categorías	CategoriaController CRUD	Listado, alta y baja
Sesiones	SesionController + CRUD	Calendario de sesiones
Asistencia / QR	AsistenciaController + token QR	Scanner QR en Angular
Usuarios	UsuarioController + roles	Gestión de usuarios y permisos

4 API REST Documentada — Swagger UI

4.1 Configuración de Swagger UI

La API del SGED está documentada mediante **SpringDoc OpenAPI 2**. Para cumplir la Directriz 3, se ajustó `application.yml` de modo que la interfaz interactiva Swagger UI quede accesible en `/api/docs`:

Listing 2: Rutas SpringDoc ajustadas para la demo

```
springdoc:
  api-docs:
    path: /api/docs.json          # JSON crudo OpenAPI 3.1
  swagger-ui:
    path: /api/docs              # Interfaz interactiva Swagger UI
    operationsSorter: method
```

4.2 Endpoints demostrados en vivo

Los tres endpoints se ejecutan en el siguiente orden durante la sesión de laboratorio.

4.2.1 Endpoint 1 — POST `/api/auth/login`

Listing 3: Request body — POST `/api/auth/login`

```
{
```



```
"username": "admin",  
"password": "Admin2026!"  
}
```

Respuesta esperada (200 OK):

Listing 4: Response — POST /api/auth/login

```
{  
  "username": "admin",  
  "role": "ADMINISTRADOR",  
  "message": "Sesion iniciada correctamente"  
}
```

La cookie `access_token` se incluye automáticamente en todas las peticiones siguientes si el manejo de cookies está activo en Postman/Insomnia.

4.2.2 Endpoint 2 — GET /api/categorias/activas

Retorna la lista de categorías activas; requiere rol ADMINISTRADOR, ENTRENADOR o RECEPCIONISTA.

Listing 5: Response — GET /api/categorias/activas (estructura CategoriaResponse.java)

```
[  
  {  
    "idCategoria": 1,  
    "nombre": "Sub-12",  
    "edadMin": 10,  
    "edadMax": 12,  
    "descripcion": "Categoria formativa sub-12",  
    "activo": true,  
    "createdAt": "2026-03-01T10:00:00Z"  
  }  
]
```

4.2.3 Endpoint 3 — POST /api/asistencias/qr/sesion/{idSesion}/token

Genera el token QR para una sesión de entrenamiento. Requiere rol ADMINISTRADOR o RECEPCIONISTA y un idSesion válido previamente creado.

Listing 6: Response — POST /api/asistencias/qr/sesion/1/token

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresAt": "2026-08-09T02:30:00Z",
  "idSesion": 1
}
```

Plan B: si no hay sesión preexistente, sustituir por POST /api/categorias (misma categoría de “endpoint que escribe”, sin dependencias de datos previos).

4.3 Checklist de ensayo previo al laboratorio

Checklist — Demo API REST en vivo

- ☐ Rutas SpringDoc ajustadas en application.yml.
- ☐ make up ejecutado; stack completo en pie.
- ☐ http://localhost:8080/api/docs abre Swagger UI (no JSON).
- ☐ Colección Postman/Insomnia guardada; cookies activas.
- ☐ Flujo completo ensayado: login → categorías activas → token QR.
- ☐ Plan B preparado (POST /api/categorias).

5 Comparativa SOAP vs. REST

5.1 Tabla de criterios

5.2 Ejemplo SOAP y su equivalente REST

5.2.1 SOAP — NumberToWords (servicio público verificado con curl)

Listing 7: Request SOAP — POST NumberToWords

Cuadro 2: Comparativa SOAP vs. REST — 9 criterios de análisis

Criterio	SOAP	REST
Formato del mensaje	XML estricto, dentro de un <i>Envelope</i> (Header + Body)	Flexible; en la práctica casi siempre JSON
Contrato del servicio	WSDL obligatorio	OpenAPI / Swagger opcional (SGED: <i>/api/docs</i>)
Transporte	Agnóstico en teoría; HTTP en la práctica	Exclusivamente HTTP con verbos semánticos
Overhead	Alto: <i>Envelope</i> XML + cabeceras WS-*	Bajo: JSON compacto, sin envoltorio obligatorio
Seguridad	WS-Security a nivel de mensaje	TLS + JWT / OAuth2 (SGED: JWT en <i>cookie HttpOnly</i>)
Manejo de errores	<i>SOAP Fault</i> estandarizado (<i>faultcode/faultstring</i>)	Códigos HTTP estándar + cuerpo de error propio
Consumo desde móvil	Pesado: parseo XML + librerías desde WSDL	Liviano: JSON nativo en cualquier SDK móvil
Caché	No aprovecha caché HTTP (todo son POST)	GET cacheables con cabeceras HTTP estándar
Casos de uso en Ecuador	Sistemas legados bancarios y gubernamentales (Garriga et al., 2021)	Apps nuevas, <i>fintechs</i> y apps móviles (incluido SGED)

```

POST https://www.dataaccess.com/webservicesserver/
    NumberConversion.wso
Content-Type: text/xml; charset=utf-8
SOAPAction: http://www.dataaccess.com/webservicesserver/
    NumberToWords

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <NumberToWords
      xmlns="http://www.dataaccess.com/webservicesserver/">
      <ubiNum>500</ubiNum>
    </NumberToWords>
  </soap:Body>
</soap:Envelope>

```

```
</soap:Body>
</soap:Envelope>
```

Listing 8: Response SOAP — captura real

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <m:NumberToWordsResponse
      xmlns:m="http://www.dataaccess.com/webservicesserver/">
      <m:NumberToWordsResult>five hundred </
        m:NumberToWordsResult>
    </m:NumberToWordsResponse>
  </soap:Body>
</soap:Envelope>
```

5.2.2 REST — Equivalente en SGED

Listing 9: Request REST — GET /api/categorias/activas

```
GET /api/categorias/activas HTTP/1.1
Host: localhost:8080
Cookie: access_token=<obtenida en POST /api/auth/login>
```

Listing 10: Response REST — array JSON directo (sin envoltorio)

```
[
  {
    "idCategoria": 1,
    "nombre": "Sub-12",
    "edadMin": 10,
    "edadMax": 12,
    "descripcion": "Categoria formativa sub-12",
    "activo": true,
    "createdAt": "2026-03-01T10:00:00Z"
  }
]
```

]

5.3 Análisis del contraste

Para devolver información útil comparable, SOAP necesita el *Envelope* XML completo (namespaces, Header/Body, nombre de la operación repetido) solo para “empacar” un resultado; REST entrega el arreglo JSON directamente. Este es el argumento empírico del criterio *overhead*: no una afirmación teórica, sino una diferencia observable en los bytes transferidos.

Pendiente (Pallo): reemplazar la respuesta de ejemplo de `/api/categorias/activas` por la captura real de Postman con datos de la base de datos del SGED antes de la entrega final.

6 Tendencias Emergentes en el Desarrollo Web

6.1 Jamstack: desacoplamiento como paradigma de arquitectura

Jamstack (*JavaScript, APIs, Markup*) es una filosofía de arquitectura web que separa radicalmente el frontend del backend: el cliente se entrega como HTML/CSS/JS pre-generado y servido desde un CDN (*Content Delivery Network*), mientras que la lógica de negocio se encapsula en APIs independientes que se consumen bajo demanda (Biilmann, 2020).

Esta separación tiene consecuencias concretas y medibles. En términos de **rendimiento**, al no existir un servidor de aplicaciones entre el usuario y el contenido estático, la latencia se reduce drásticamente; plataformas como Netlify y Vercel reportan tiempos de primer byte (TTFB) por debajo de 50 ms para sitios Jamstack distribuidos globalmente. En términos de **seguridad**, la superficie de ataque se reduce: sin servidor de aplicaciones expuesto, vulnerabilidades típicas como inyección SQL o SSRF se mitigan por diseño. En términos de **escalabilidad**, el CDN escala automáticamente ante picos de tráfico sin configuración adicional.

Frameworks representativos de este paradigma incluyen *Next.js* (React, híbrido SSR/SSG), *Nuxt* (Vue.js), *Astro* (agnóstico de framework, optimizado para contenido) y *SvelteKit*.

Sin embargo, Jamstack no es una solución universal y su adopción acrítica es un error frecuente. Aplicaciones con alta carga de datos en tiempo real — paneles de administración, sistemas de asistencia como el SGED, plataformas de comunicación — se benefician más de un backend tradicional con WebSockets o SSE (*Server-Sent Events*) que de un generador de sitios estáticos. La elección del paradigma debe responder al perfil de uso real de la aplicación, no a tendencias de mercado.

6.2 Progressive Web Apps (PWA): cierre de la brecha web/nativa

Una *Progressive Web App* es una aplicación web que aprovecha APIs modernas del navegador para ofrecer una experiencia equivalente a la de una app nativa: instalación en pantalla de inicio, funcionamiento offline mediante *Service Workers*, notificaciones *push* y acceso a hardware del dispositivo (cámara, GPS, acelerómetro) (MDN Web Docs, 2023).

El impacto en el contexto latinoamericano es especialmente relevante. Según datos de GSMA Intelligence, más del 60 % de los usuarios móviles en Ecuador acceden a Internet desde dispositivos de gama media-baja con conectividad intermitente. Una PWA que funcione offline y cargue en menos de 3 segundos en una red 3G tiene una ventaja competitiva sustancial frente a una app nativa que requiere descarga desde una tienda de aplicaciones y espacio de almacenamiento adicional.

El SGED ya incorpora esta tecnología: el frontend Angular incluye el paquete `@angular/service-worker` y una configuración activa en `frontend/ngsw-config.json`.

6.2.1 Demo funcional — Service Worker en el SGED

El archivo `ngsw-config.json` define dos estrategias de caché complementarias alineadas con el perfil de uso real de la aplicación:

Listing 11: Estrategias de caché del Service Worker (`frontend/ngsw-config.json`)

```
{
  "dataGroups": [
    {
      "name": "catalogos",
      "urls": ["/api/categorias/activas", "/api/
        estados_generales"],
    }
  ]
}
```

```
    "cacheConfig": {
      "strategy": "performance",
      "maxSize": 20,
      "maxAge": "1d"
    }
  },
  {
    "name": "sesiones",
    "urls": ["/api/evaluaciones/sesion/**"],
    "cacheConfig": {
      "strategy": "freshness",
      "timeout": "3s",
      "maxSize": 30,
      "maxAge": "12h"
    }
  }
],
"navigationRequestStrategy": "performance"
}
```

La estrategia *performance* (caché primero) aplica a los catálogos — categorías activas y estados generales — que rara vez cambian: la pantalla abre al instante aunque no haya red disponible. La estrategia *freshness* (red primero con *timeout* de 3 segundos) aplicada a los datos de evaluaciones por sesión refleja una decisión de diseño concreta: la asistencia varía durante el entrenamiento, por lo que es preferible intentar la red primero; pero si tarda más de 3 segundos — escenario habitual en canchas deportivas con cobertura débil — el Service Worker devuelve el último valor conocido en lugar de dejar al entrenador esperando con la pantalla en blanco. Esta combinación hace que el SGED funcione de forma utilizable en condiciones de conectividad intermitente sin ningún cambio en la lógica de la aplicación.

6.3 Inteligencia Artificial Generativa: reconfiguración del rol del desarrollador

La irrupción de modelos de lenguaje de gran escala (LLM) como GPT-4o, Claude 3.5 y Gemini 1.5 Pro está transformando el flujo de trabajo del desarrollador web de formas que van mucho más allá de la autocompletación de código (GitHub, 2023).

Cuatro vectores de impacto son especialmente visibles:

1. **Generación de código y pruebas.** Herramientas como GitHub Copilot, Cursor y Tabnine generan implementaciones completas a partir de descripciones en lenguaje natural, reduciendo el tiempo de prototipado en un 40–55 % según estudios de productividad de GitHub (2023) (GitHub, 2023).
2. **Revisión automática de código.** Modelos integrados en pipelines CI/CD identifican vulnerabilidades de seguridad, violaciones de estilo y posibles regresiones antes de que el código llegue a producción.
3. **Generación de documentación.** La especificación OpenAPI del SGED podría generarse automáticamente a partir del código fuente mediante LLMs *fine-tuned*, eliminando la brecha habitual entre código y documentación.
4. **Interfaces conversacionales.** La integración de chatbots en aplicaciones web (mediante APIs como OpenAI o Google Generative AI) permite interacciones en lenguaje natural que sustituyen formularios complejos en casos de uso de baja frecuencia.

Posición crítica. La IA generativa no elimina la necesidad de comprensión profunda del sistema. El código generado por LLMs puede contener errores sutiles, alucinaciones (referencias a funciones inexistentes) y vulnerabilidades que solo un desarrollador con criterio técnico sólido puede detectar y corregir. El riesgo real no es que la IA reemplace a los desarrolladores, sino que desarrolladores sin suficiente criterio crítico deleguen decisiones de arquitectura y seguridad a modelos que no tienen contexto del dominio de negocio. La profesión no desaparece; se desplaza hacia la validación, la especificación de requisitos precisos y el diseño de sistemas donde la IA actúa como herramienta, no como árbitro de decisiones.

En el contexto educativo ecuatoriano, esto implica que la formación en fundamentos — arquitecturas, protocolos, seguridad, bases de datos — sigue siendo indispensable: no para competir con la IA, sino para usarla con rigor y responsabilidad.

7 Conclusiones

1. **REST como estándar consolidado.** La disertación de Fielding (2000) sigue siendo el fundamento teórico irrefutable de las APIs modernas. El SGED implementa sus restricciones de forma coherente, lo que se traduce en una API comprensible, extensible y documentada mediante OpenAPI 3.1 / Swagger UI.
2. **SOAP: contexto, no obsolescencia.** SOAP coexiste con REST en entornos donde la integridad del mensaje a nivel protocolar es crítica (banca, servicios gubernamentales). En Ecuador, la transición en el sector público avanza lentamente por razones de compatibilidad histórica (Garriga et al., 2021), lo que hace relevante conocer ambos paradigmas.
3. **MVC como arquitectura estructurante.** El patrón MVC, implementado con Spring Boot y React, demostró ser suficientemente flexible para soportar los módulos de asistencia QR, gestión de categorías y control de usuarios del SGED, manteniendo una separación clara de responsabilidades.
4. **Tendencias como oportunidad, no amenaza.** Jamstack, PWA e IA generativa complementan y amplifican las competencias técnicas fundamentales. Un equipo que entiende los protocolos subyacentes puede aprovechar estas tecnologías con criterio; uno que no los entiende, construye sobre cimientos que no comprende.
5. **Documentación como ciudadano de primera clase.** La integración de SpringDoc en el SGED convirtió la documentación en una consecuencia automática del código (*doc-as-code*), reduciendo la divergencia entre la API real y su descripción formal.

8 Trabajo Futuro

- **PWA — registro offline de asistencias:** el SGED ya cuenta con un Service Worker activo (`ngsw-config.json`) orientado a caché de catálogos y sesiones.

El siguiente paso es extenderlo con una cola de escritura offline que permita registrar asistencias sin conexión y sincronizarlas con el servidor al recuperar señal, completando así el ciclo PWA para el flujo de entrenamiento.

- **GraphQL como capa alternativa:** exponer un endpoint GraphQL sobre los mismos servicios del SGED para permitir consultas flexibles desde el frontend sin *over-fetching* ni *under-fetching*.
- **Generación de pruebas con IA:** integrar un agente basado en LLM que, a partir de la especificación OpenAPI, genere automáticamente casos de prueba de integración para el backend.
- **Migración a OAuth2 / OIDC:** reemplazar el JWT artesanal actual por un proveedor de identidad estándar (Keycloak, Auth0) para facilitar la integración con sistemas universitarios externos.
- **Análisis de rendimiento Jamstack:** prototipar el frontend con Next.js en modo SSG/ISR para comparar métricas de Core Web Vitals contra la versión SPA actual.

Referencias

- Biilmann, M. (2020). *The JAMstack origin story*. Consultado el 1 de agosto de 2026, desde <https://www.netlify.com/blog/2020/04/14/what-is-a-static-site-generator/>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Garriga, M., Mateos, C., Flores, A., Cechich, A., & Zunino, A. (2021). RESTful service composition at a glance: A survey. *Journal of Network and Computer Applications*, 60, 32-53. <https://doi.org/10.1016/j.jnca.2015.11.020>
- GitHub. (2023). *The economic impact of the AI-powered developer lifecycle* (inf. téc.). GitHub, Inc. <https://github.blog/2023-06-27-the-economic-impact-of-the-ai-powered-developer-lifecycle/>
- Masse, M. (2011). *REST API Design Rulebook*. O'Reilly Media.
- MDN Web Docs. (2023). *Progressive web apps (PWAs)*. Consultado el 1 de agosto de 2026, desde https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- SpringDoc Team. (2024). *SpringDoc OpenAPI 2 — Reference Documentation*. Consultado el 1 de agosto de 2026, desde <https://springdoc.org/v2/>
- W3C. (2007). *SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)* (inf. téc.). World Wide Web Consortium. <https://www.w3.org/TR/soap12-part1/>

A Archivo `referencias.bib`

A continuación se incluye el contenido completo del archivo `referencias.bib` que debe existir en la misma carpeta que este documento para que la compilación con `biber` funcione correctamente.

```
@phdthesis{fielding2000,
  author      = {Fielding, Roy Thomas},
  title       = {Architectural Styles and the Design of
                 Network-based Software Architectures},
  school      = {University of California, Irvine},
```

```
    year      = {2000},
    url       = {https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
    }
}

@book{richardson2007,
  author      = {Richardson, Leonard and Ruby, Sam},
  title       = {RESTful Web Services},
  publisher    = {O'Reilly Media},
  year        = {2007},
  isbn        = {978-0-596-52926-0}
}

@online{springdoc2024,
  author      = {{SpringDoc Team}},
  title       = {SpringDoc OpenAPI 2 --- Reference Documentation},
  year        = {2024},
  url         = {https://springdoc.org/v2/},
  urldate     = {2026-08-01}
}

@techreport{w3c2007,
  author       = {{W3C}},
  title        = {SOAP Version 1.2 Part 1: Messaging Framework
    (Second Edition)},
  institution  = {World Wide Web Consortium},
  year         = {2007},
  url          = {https://www.w3.org/TR/soap12-part1/}
}

@book{masse2011,
  author       = {Masse, Mark},
  title        = {REST API Design Rulebook},
  publisher     = {O'Reilly Media},
  year         = {2011},
  isbn         = {978-1-4493-1050-9}
}

@article{garriga2021,
```

```
author  = {Garriga, Martin and Mateos, Cristian and  
          Flores, Alejandro and Cechich, Alejandra and  
          Zunino, Alejandro},  
title   = {RESTful service composition at a glance:  
          A survey},  
journal = {Journal of Network and Computer Applications},  
year    = {2021},  
volume  = {60},  
pages   = {32--53},  
doi     = {10.1016/j.jnca.2015.11.020}  
}  
  
@online{biilmann2020,  
  author = {Biilmann, Mathias},  
  title  = {The JAMstack origin story},  
  year   = {2020},  
  url    = {https://www.netlify.com/blog/2020/04/14/  
            what-is-a-static-site-generator/},  
  urldate = {2026-08-01}  
}  
  
@online{mozilla2023,  
  author = {{MDN Web Docs}},  
  title  = {Progressive web apps (PWAs)},  
  year   = {2023},  
  url    = {https://developer.mozilla.org/en-US/docs/Web/  
            Progressive_web_apps},  
  urldate = {2026-08-01}  
}  
  
@techreport{github2023,  
  author      = {{GitHub}},  
  title       = {The economic impact of the AI-powered  
                developer lifecycle},  
  institution = {GitHub, Inc.},  
  year        = {2023},  
  url         = {https://github.blog/2023-06-27-the-economic-impact-  
                of-the-ai-powered-developer-lifecycle/}  
}
```